

FUNCTIONS

FUNCTIONS

```
function function_name(){  
}
```

FUNCTIONS-types

1. Anonymous function
2. arrow function
3. Call Back Functions
4. Higher-Order Function
5. Pure Functions

1. PURE FUNCTION ASSIGNMENT

ex 1: Write a pure function that takes a number and returns its cube.

Solution:

```
function cube(n) {  
    return n * n * n;  
}
```

```
console.log(cube(3));
```

ex 2: Write a pure function that takes an array of numbers and returns a new array with all numbers doubled. The

Solution:

```
function doubleArray(arr) {  
    return arr.map(n => n * 2);  
}
```

```
const numbers = [1, 2, 3, 4];  
const doubled = doubleArray(numbers);
```

```
console.log(doubled);  
console.log(numbers);
```

ex 3: Create a pure function called filterAndMultiply that takes an array of numbers and a multiplier as argument

1. Filter: Remove any numbers in the array that are less than 5.
2. Multiply: Multiply the remaining numbers by the provided multiplier.
3. Return: Return a new array with the filtered and multiplied values.

solution:

```
function filterAndMultiply(numbers, multiplier) {  
    return numbers  
        .filter(n => n >= 5)  
        .map(n => n * multiplier);  
}
```

```
const data = [3, 5, 8, 2, 10, 1];  
const result = filterAndMultiply(data, 3);
```

```
console.log(result);  
console.log(data);
```

2. Anonymous functions

ex 1:

Create an anonymous function that calculates the area of a rectangle. The function should take two parameters:

Solution:

```
const area = (function(width, height) {  
    return width * height;  
})(10, 5);
```

```
console.log(area);
```

ex 2: you can try different examples or assignments by your-self.

3.ARROW FUNCTIONS

ex 1:

Use an arrow function inside the map method to create a new array where each number from the original array is

Solution:

```
const numbers = [1, 2, 3, 4];
const squaredNumbers = numbers.map(n => n * n);

console.log(squaredNumbers);
```

ex 2:

Create an array of objects representing products in a store. Each product object should have properties: name, price, and quantity. Use arrow functions to:

1.Filter: Create a new array containing only the products with a price greater than \$20.

2.Calculate: For each product in the filtered array, calculate the total value (price multiplied by quantity).

solution:

```
const products = [
  { name: 'Laptop', price: 999, quantity: 1 },
  { name: 'Mouse', price: 25, quantity: 3 },
  { name: 'Keyboard', price: 45, quantity: 2 },
  { name: 'Monitor', price: 150, quantity: 1 },
  { name: 'USB Cable', price: 10, quantity: 5 }
];

const expensiveProducts = products.filter(product => product.price > 20);
const totalValues = expensiveProducts.map(product => product.price * product.quantity);

console.log(expensiveProducts)
console.log(totalValues);
```

4.CALLBACK FUNCTIONS

ex 1:

Write a function called processNumber that takes a number and a callback function. The function should call the

Solution:

```
function processNumber(num, callback) {
  callback(num * 2);
}

processNumber(5, function(result) {
  console.log(result);
});
```

ex 2:

Create a function performOperation that takes two numbers and a callback function. The callback should perform

Solution:

```
function performOperation(num1, num2, callback) {
  return callback(num1, num2);
}

const add = (a, b) => a + b;

const result = performOperation(8, 4, add);
console.log(result);
```

5.HIGHER-ORDER FUNCTIONS

ex 1:

Write a higher-order function filterNumbers that takes an array of numbers and a square. The filterNumbers func

Solution:

```
function filterNumbers(arr, callback) {
  return arr.filter(callback);
}

function isEven(num) {
  return num % 2 === 0;
}

const numbers = [1, 2, 3, 4, 5, 6];
const evenNumbers = filterNumbers(numbers, isEven);

console.log(evenNumbers);
```

Ex 2:

dialogue prompt- alert

browser- document.write

console- console.log

Create a higher-order function createMultiplier that returns a function to multiply a given number by a specific

Solution:

```
function createMultiplier(factor) {
  return function(number) {
    return number * factor;
  };
}

const double = createMultiplier(2);

const numbers = [1, 2, 3, 4, 5];
const doubledNumbers = numbers.map(double);

console.log(doubledNumbers);
```